Constructing objectKV: One Transaction History over Memory, Stable Logs, and Immutable Objects

Wiley Jones
DOSS
New York, USA
wiley@doss.com

ABSTRACT

objectKV is an open ordered transactional key-value kernel for object-native data platforms. It separates three concerns that cloud databases often bind together: transaction ordering, foreground serving, and permanent state. A bounded cell orders and conflict-checks transactions. A replicated stable-media transaction log acknowledges durable commits without waiting for object storage. Disposable RangeEngines serve assigned ranges from RAM, native NVMe, or RocksDB. Background materializers publish authenticated immutable objects that support reconstruction, branching, row access, and columnar scans.

This paper describes the target architecture and the measured construction boundary. The current implementation verifies a topology-matched single-range RocksDB read path within 12.7% of direct RocksDB throughput and within 18.4% of its p99 through 32 clients. A generation-pinned GCS layout preserved point p99 while improving a projected scan by 31.692x in a scoped preflight. Complete GCS recovery reconstructed 25,014 MVCC records with five named GETs and rejected a one-byte corruption. A TLA+ model explored 264,076 distinct states across two finite scopes; six weakened-contract controls each reached its named invariant violation. These results do not establish a complete cell. Independent-machine quorum latency, multi-range transactions, RAM admission, PostgreSQL, Redis, and scaled HTAP remain open gates. The contribution is therefore a precise system shape, evidence boundary, and evaluation program for deciding each layer with matched controls.

PVLDB Reference Format:
Wiley Jones. Constructing objectKV: One Transaction History over Memory, Stable Logs, and Immutable Objects. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/Doss-com/objectKV.

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

1 INTRODUCTION

Object storage is an effective capacity and history substrate, but it is a poor default coordination primitive for latency-sensitive multi-writer transactions. Point reads also cannot depend on a full restore or an object request per key. A practical object-native database therefore still needs a fast foreground path. The architectural question is not whether local state exists. It is whether local state remains the permanent database, or becomes a bounded and replaceable accelerator over a reconstructable object base.

objectKV takes the second position. One cell provides a strict-serializable ordered KV contract across its ranges. A stable replicated *txLog* protects acknowledged mutations. RangeEngines materialize only their assigned working sets. Immutable object closures hold the portable base, retained history, branch roots, and scan-oriented projections. PostgreSQL pages, Redis-style objects, ordered logs, search indexes, virtual filesystems, and DataFusion tables are intended to meet at one narrow `okv-fabric` boundary rather than owning separate consistency domains.

The design is object-native but not object-only. This distinction follows the same broad physical observation made by cloud database systems such as Aurora, Socrates, and Neon: log, storage, and compute can be separated without placing every foreground operation on the capacity tier [1, 11, 18]. It also adopts FoundationDB's small ordered transaction API and bounded transaction domain as the semantic target [20], while retaining TiKV and RocksDB as important physical controls [7, 8].

The paper makes four contributions:

- A bottom-up architecture that separates immutable state, stable commit history, disposable serving compute, cell transactions, and typed APIs.
- One object layout that aligns row identity, opaque values, columnar projection, integrity, and exact recovery without creating two database truths.
- An executable cell safety model with explicit commit, generation, publication, reclamation, and serving invariants.
- A governed performance matrix that reports scoped receipts and open gates rather than treating code presence as system evidence.

2 REQUIREMENTS AND STATUS SEMANTICS

2.1 Product requirements

Table 1 fixes the current system intent. The kernel is not a SQL engine, a Redis clone, or a global metacluster. Those are fabric surfaces or fleet concerns above a bounded transaction cell.

Table 1: System requirements and explicit non-goals.

Concern	Requirement	Deliberate boundary
Semantics	Ordered binary keys, point and range reads, snapshots, atomic mutation, short strict-serializable transactions	No synchronous transaction across cells
Commit	A durable reply requires a declared stable quorum; object publication is asynchronous in the default profile	Object storage is not the default consensus or acknowledgement service
Serving	Assigned ranges are fast from bounded RAM, native NVMe, or RocksDB state	No serving worker owns the only copy of acknowledged data
Persistence	Immutable, generation-pinned S3-compatible closures are sufficient for reconstruction with a retained log suffix	No LIST-based discovery of authoritative current state
Analytics	One snapshot version feeds row reads and exact columnar base-plus-tail queries	Columnar materialization lag may raise query cost, but cannot silently lower freshness
Portability	Open object layouts and an engine-neutral fabric permit independent compute and storage evolution	No requirement that every hot provider share one physical file format
Operations	Cells bound failure radius, recovery generation, version space, and capacity	No first-release global distributed transaction or autonomous metacluster

2.2 Evidence vocabulary

The manuscript uses four statuses. [Code Complete] means the implementation exists and its focused checks pass. [Verified] means a named claim has a retained receipt that binds source, workload, topology, control, evaluator, and metrics. [Evaluating] means the implementation or evidence is under active verification. [Proposed] and [Future] describe design or later scope. Every status is local to the stated claim. A verified single-range read does not verify a cell.

3 ARCHITECTURE FROM STORAGE UPWARD

Figure 1 is the compact system map. Permanent bytes sit at the bottom. Transactions and serving are rebuilt above those bytes. Typed interfaces sit at the top.

3.1 Immutable object closure

The permanent base is a typed root naming a complete immutable closure. The root binds a generation, child inventory, format versions, logical bounds, and cryptographic identities. Readers open exact names. Writers create new objects and atomically activate a new root through the transaction authority. Old roots remain available while branches, snapshots, backups, queries, or recovery leases reference them.

Let C be the latest committed version and O the active object frontier. The core recovery relationship is:

$$O \le C, \qquad Database(C) = ObjectState(O) + StableTxLog(O, C]. \tag{1}$$

The object closure need not match the in-memory representation. It must preserve ordered key identity, MVCC history required by policy, integrity, range bounds, and enough metadata to reconstruct a serving image without searching the bucket.

3.2 Stable transaction log

The *txLog* is the authoritative suffix of committed mutation outcomes after O. It is not a time log and it is not the application-facing `okv-log`. In the default durable profile, a commit is acknowledged only after a quorum of independent stable media accepts it. Object publication may continue later. The log prefix can be reclaimed only after the corresponding object closure is complete, authenticated, active in the same generation, and protected by every relevant root.

Two higher-level log abstractions build above this boundary:

`okv-log` An append, read, trim, replay, and branch API for applications. Records may contain deterministic semantic operations.

`okv-wal` A recovery-oriented library built on `okv-log`. It adds durable positions, replay rules, checksums, and checkpoint references.

Neither abstraction replaces the physical committed-mutation *txLog* until crash injection proves equivalent outcomes across software versions.

3.3 RangeEngine serving compute

A RangeEngine is the disposable compute runtime for an assigned ordered range. It owns a recent MVCC overlay, coverage metadata, one chosen resident engine, read and write budgets, and materialization responsibility. It may use:

- `ram_image`: an admitted record, block, or complete-range image in DRAM;
- `native_nvme`: a future append-oriented hybrid log and ordered index on local NVMe;
- `rocksdb_image`: a mature disposable ordered image and the present verified single-range provider.

These are provider choices, not three database truths. A range uses one primary resident provider at a time, with optional bounded cache above it and an indexed object miss path below it. Capacity is bounded by assignment and admission. When the high watermark is reached, the control plane evicts cache, demotes ranges, splits assignments, or adds workers. Local bytes should scale with the assigned working set, not total database bytes.

Serving tier and commit depth are independent axes. A RAM image can use quorum durability; a RocksDB image can run in a relaxed local-only profile; an object-gated profile can wait for

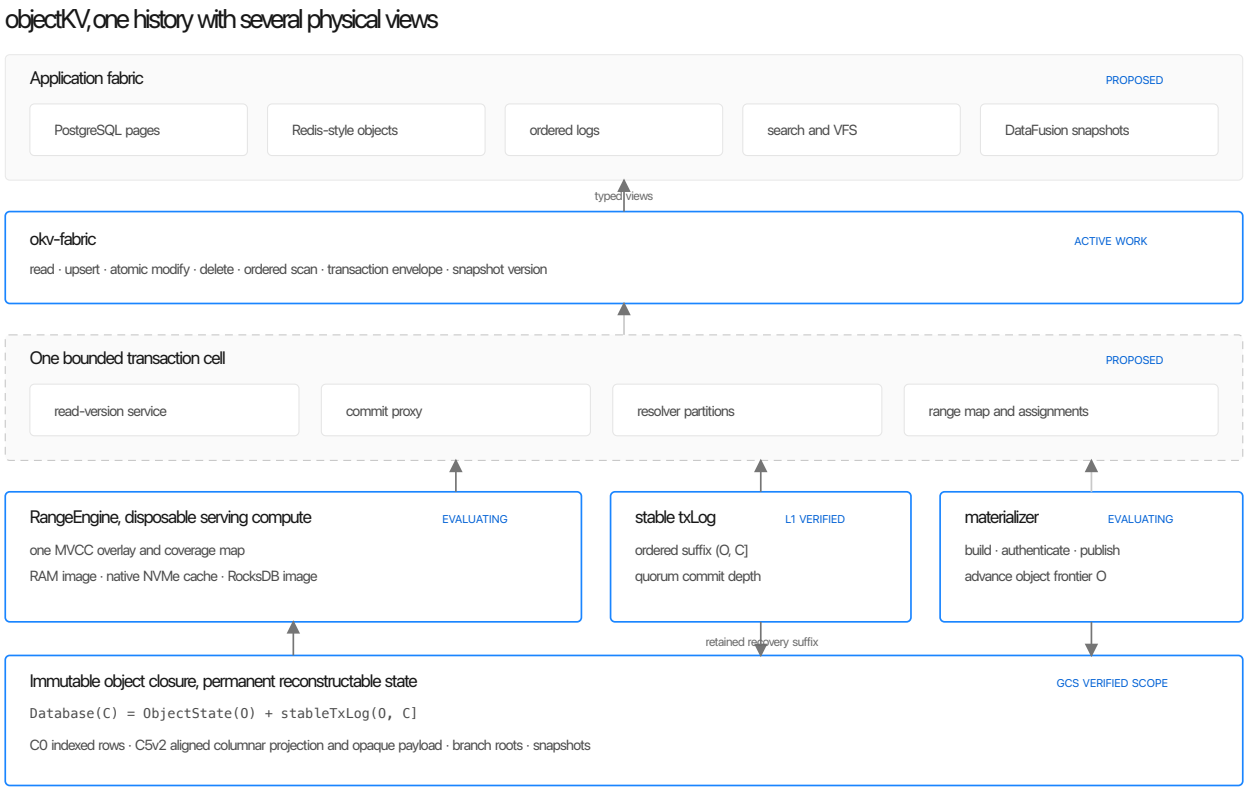

Figure 1: objectKV is one version history with several physical views. Solid blue boundaries have scoped implementation evidence. Dashed boundaries are target structure.

publication. A configuration must name both its latency shape and its loss envelope.

Garnet reinforces two aspects of this design. Its current implementation opens typed string, object, unified, and vector sessions over one TsavoriteKV per logical database, rather than one consistency domain per API [5]. Its tiered device also separates hot-to-cold order from the commit point. objectKV adopts those structural lessons, but not Garnet's asynchronous replication or hash-slot transaction boundary.

3.4 One bounded transaction cell

A cell is the distributed system, not a page, object, block, or shard. Figure 2 shows the target service topology. One cell has one commit version space, recovery generation, range map, and stable-log contract. A tenant transaction may touch arbitrary ranges inside its cell. Cells do not synchronously coordinate transactions with each other.

The target cell separates five service classes:

(1) read-version services choose a snapshot version;
(2) commit proxies batch transactions and assign ordered commit work;
(3) resolver partitions validate read and write conflict ranges;
(4) *txLog* sets make committed outcomes durable;
(5) RangeEngines serve and materialize assigned key ranges.

Generation, membership, assignment, recovery, and rate-limit authorities govern those services. Initial implementations may co-locate roles, but interfaces must preserve the separation so measured bottlenecks can later be partitioned.

3.5 okv-fabric

The fabric is the shared application boundary over one RangeEngine state. Its candidate narrow waist is:

```
read, upsert, atomic_modify, delete, ordered_scan
```

Operations run inside an explicit transaction or snapshot envelope. Typed views compile above the waist: bytes and pages for PostgreSQL, structured values for a Redis-like surface, ordered records for logs and inverted indexes, inode and extent records for a virtual filesystem, and Arrow batches for analytical compute. Direct session-to-engine execution is a candidate fast path. Per-core dispatch remains an alternative that must justify its queue, copy, and context switch costs.

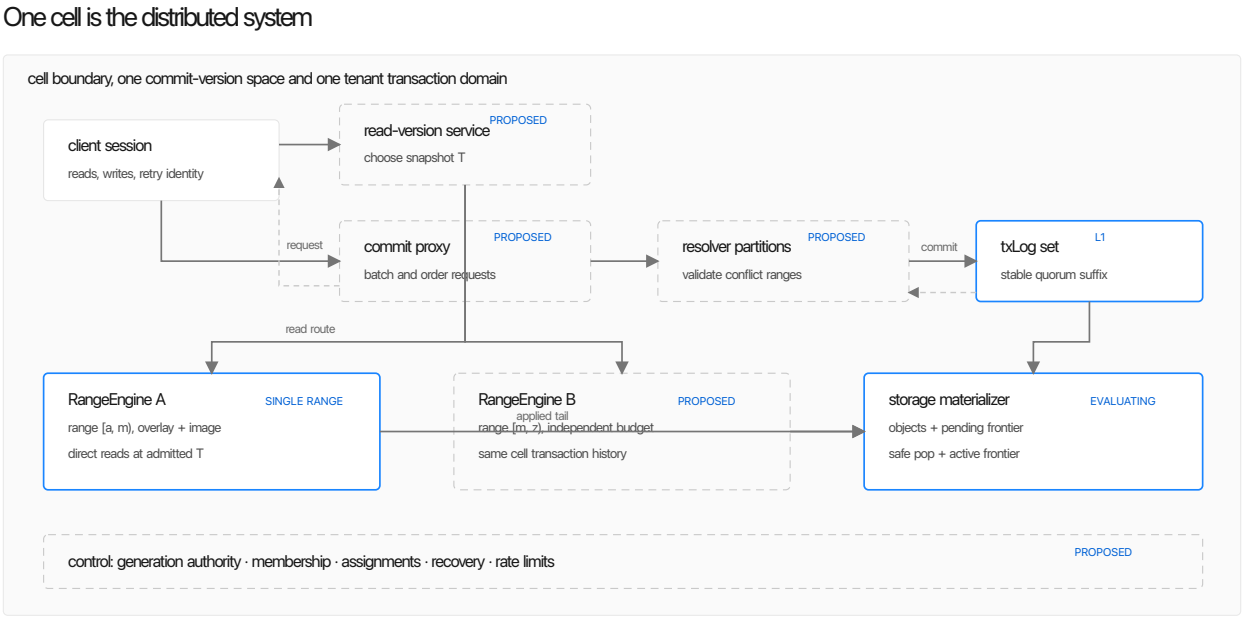

Figure 2: Target service taxonomy inside one cell. The current implementation has single-range serving and one-host log mechanics, not this complete distributed topology.

4 FOREGROUND AND BACKGROUND PROTOCOLS

Figure 3 places commit, point-read, and analytical work side by side. A useful implementation keeps their foreground dependencies distinct.

4.1 Commit

A transaction reads at version T, records read conflict ranges, and submits a bounded mutation set plus an idempotency identity. The commit proxy assigns ordered work. Resolvers reject a transaction if a committed write after T intersects its read conflict ranges. Accepted mutations receive a unique version, persist to the required *txLog* set, and only then return `COMMITTED`. RangeEngines apply tagged mutations asynchronously or from their local log stream. A lost reply is resolved by transaction identity, not by blindly applying the transaction again.

The current TLA+ model captures this order. The complete multi-range protocol, partitioned conflict representation, and independent-media performance remain [Evaluating] or [Proposed].

4.2 Latest point read

A latest read chooses T, routes to the range assignment, checks the mutable overlay, and then consults coverage metadata before the resident provider. A negative lookup is valid only when the provider proves complete coverage through T. Otherwise the RangeEngine follows the generation-pinned root and compact index to fetch the required immutable frame. It then fills the admitted cache or resident image without changing logical state.

The fast path is therefore:

$$Read(k, T) = NewestVisible(Overlay(k, T), Resident(k, T)). \quad (2)$$

An object lookup is an explicit miss path. It is not hidden inside every read.

4.3 Publication and recovery

A materializer freezes a range interval through version P, emits immutable children, verifies their hashes and logical digest, prepares a frontier update, and activates it only in the current generation. Log reclamation follows activation, not upload completion alone. A replacement worker starts from the active root, hydrates enough state for its assigned range, and replays the retained suffix through C.

Unknown object outcomes are resolved by exact named reads and content digest. Bucket listing is not an authority. A stale generation cannot publish, pop a log prefix, or serve a latest read.

4.4 Exact HTAP snapshot

For analytical partition p, let W_p be its columnar base watermark and T the query snapshot. Exact state is:

$$Table_p(T) = ColumnarBase_p(W_p) + RowChanges_p(W_p, T]. \quad (3)$$

The tail is reduced to the latest mutation per primary key. A streaming merge suppresses base rows touched by the tail, emits surviving tail upserts, and drops deletes. Tail keys are required for invalidation even if their new rows do not satisfy the final

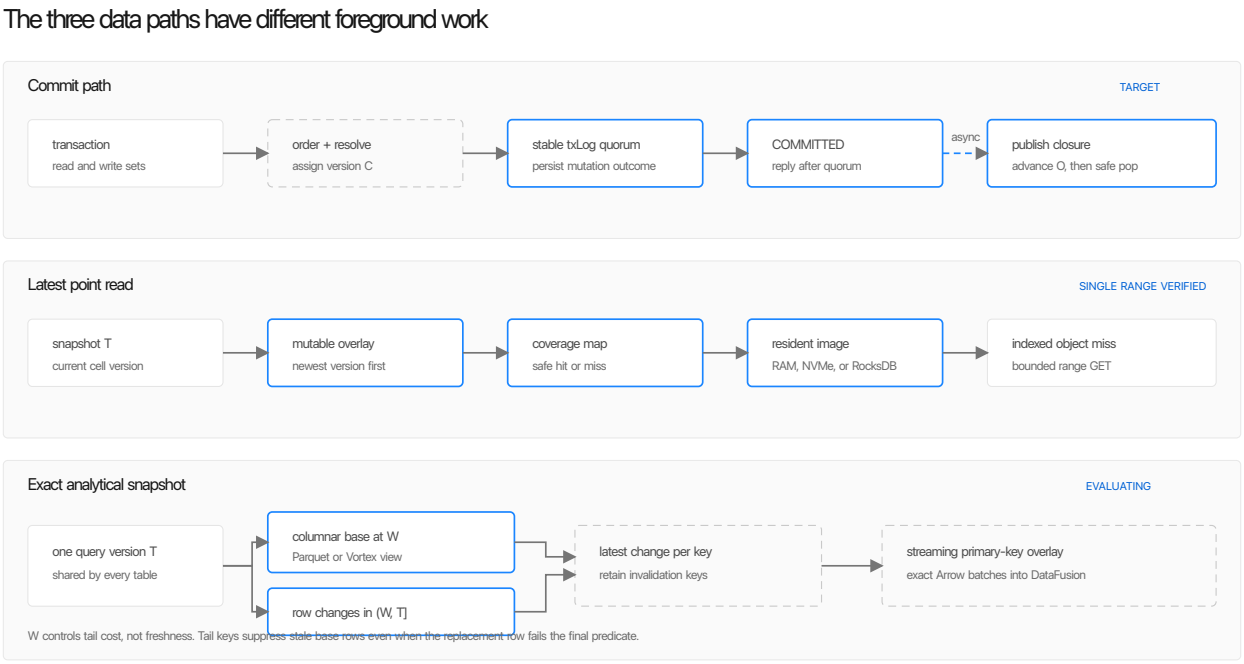

Figure 3: Foreground dependencies for the three main data paths. The default commit does not wait for object publication. A resident point hit issues no object request. An analytical query combines a column base with an exact invalidating tail.

predicate. Predicate pushdown that removes those keys is incorrect. DataFusion can consume the resulting ordered Arrow batches through a custom table provider and execution plan [9, 10].

Every table in one query uses the same T, although partitions may have different W_p. The watermark controls query cost, not freshness. Long queries hold immutable-object and delta-retention leases, not open OLTP transactions. Values used to enforce a frequent transactional invariant should instead be maintained as transactional projections. Broad ad hoc analytical decisions require dependency tokens and short revalidation before writing.

5 C5V2 OBJECT LAYOUT

A pure row layout makes projected scans fetch opaque values. A pure column layout can make point reconstruction and high-update MVCC expensive. C5v2 keeps one authoritative closure but aligns two physical streams: a projection stream with ordered keys and selected columns, and a payload stream with opaque values and tombstones. The compact index maps keys and ranges to aligned frame groups.

Point lookups use the compact index and fetch the required aligned frame pair. Projection scans coalesce range requests and avoid payload bytes. Complete recovery fetches and authenticates every named child. A frame contains enough version and key identity to reproduce the logical MVCC oracle. This approach resembles the broader HTAP lesson from HANA that columnar primary storage can serve transactional work when delta handling and access paths are designed together [15, 16]. It does not imply that the present layout is ready to replace a resident point engine.

The layout is format-pluggable above its logical contract. Parquet is a broad interchange and scan format [4]; Vortex and Lance explore lower-latency random access and richer structural encodings [13, 19]. Selection must follow a matched point, scan, compaction, and recovery curve, not format preference.

6 EXECUTABLE SAFETY CONTRACT

6.1 Model boundary

`ObjectKVCell.tla` is the compact protocol contract for one cell. Its actions cover transaction begin and sequencing, RAM staging, stable quorum persistence, commit delivery, generation change, immutable closure construction, frontier preparation and activation, safe *txLog* pop, media loss, image hydration, and reads.

The main invariants are:

- `RepliesTellTheTruth`: committed replies require stable quorum;
- `GenerationsAreFenced`: stale generations cannot commit or serve;
- `PublicationsAreComplete`: active roots name complete closures;
- `SafeTxLogPop`: reclaimed history is protected by the active object frontier;

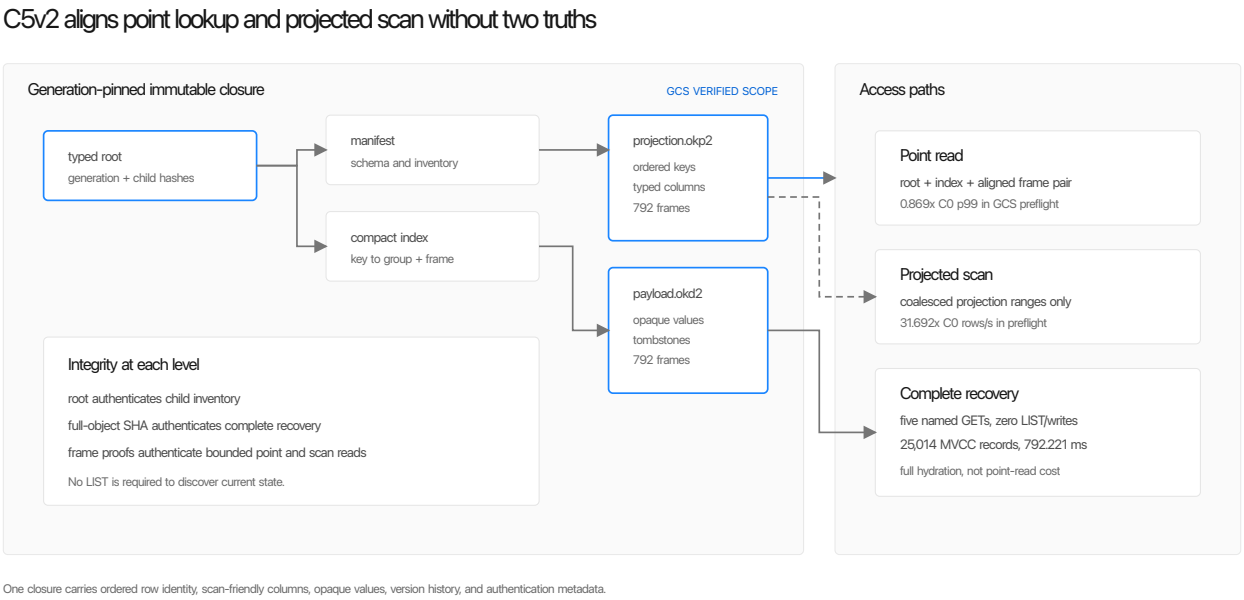

Figure 4: The C5v2 closure supports bounded point reads, projection-only scans, and complete authenticated reconstruction.

- `ServingReadsAreSafe`: latest reads use a reconstructable, current-generation image;
- `ConflictsAreValidated`: conflicting transactions do not both commit.

The model abstracts consensus to quorum intersection and generation fencing. It does not prove Raft, Paxos, object storage, RocksDB, liveness, unbounded scale, or SQL semantics. This composition follows Oswald's useful practice of modeling log, snapshots, garbage collection, and concurrent actors together, while using a different acknowledgement boundary [17].

6.2 Finite-state results

Table 2 reports TLC 2.19 using `tla2tools` 1.7.4 for model SHA-256 `55d5bb137b9e...`. TLC found no invariant violation in the two named finite scopes. Every weakened-contract control reached its exact named invariant violation and did not terminate on a parser, semantic, or incomplete successor error.

A hand-written Rust trace checker replays selected events, verifies the model identity, and validates constant assumptions. It is [Code Complete], not a mechanical refinement mapping. A current-model implementation trace and a mechanically checked action-to-event relation are required before calling Rust conformance verified. Liveness and unbounded correctness remain [Evaluating].

7 IMPLEMENTATION AND INFRASTRUCTURE BOUNDARY

Figure 5 separates mechanism evidence from operational evidence. The project has used real files, MinIO, TCP processes, process kill, GCP local NVMe, regional GCS, restricted service accounts, and independent OTel exports. It has not yet run the native commit path across three independent machines and media.

8 EVALUATION

8.1 Method

Every admitted point names the source and evaluator binary, dataset, payload distribution, warm or cold state, concurrency, storage devices, object-store authority, durability depth, process order, control, and OTel identity. AB and BA ordering limits process-order bias. Negative controls corrupt or weaken one specific contract and must fail at the named boundary. Ratios are reported only against matched semantics and topology.

The present results measure isolated mechanisms. They must not be multiplied or averaged into a complete-stack score.

8.2 Resident hot read

The admitted native provider and direct RocksDB control both return owned values inside the same recovered topology. At one client, native throughput ratios were 0.909x and 0.920x; p99 was 0.913x in both orders. At 8 clients, throughput was 0.880x and 0.873x; p99 was 1.184x and 1.122x. At 32 clients, throughput was 0.880x and 0.891x; p99 was 1.107x and 1.148x. All points passed the declared 0.80 throughput and 1.20 p99 bounds and issued zero measured object operations.

This result says the RangeEngine abstraction can preserve local ordered-read performance at one bounded scale. It says nothing about replicated commit, multi-range routing, eviction, or SQL execution.

Table 2: Executable model results. Healthy scopes are exhaustive only within the listed constants.

Configuration	Scope or expected violation	Generated	Distinct	Depth	Result
Integrated	3 nodes, 1 transaction, 2 generations, 1 media failure	2,484,568	99,408	23	[Verified]
Concurrency	3 nodes, 2 conflicting transactions, 2 generations	4,496,463	164,668	28	[Verified]
Ack before quorum	`RepliesTellTheTruth`	157	71	5	poison hit
Ignore generation fence	`GenerationsAreFenced`	10,793	1,468	9	poison hit
Pop stale frontier	`SafeTxLogPop`	82,551	7,963	12	poison hit
Publish incomplete closure	`PublicationsAreComplete`	13,418	1,738	9	poison hit
Serve stale image	`ServingReadsAreSafe`	1,483	302	6	poison hit
Skip conflict validation	`ConflictsAreValidated`	220,294	15,475	15	poison hit

Figure 5: The current evidence ladder. Higher rungs do not inherit lower-rung claims unless one receipt composes them.

Table 3: Current implementation boundary by subsystem.

Subsystem	Status	Current evidence	Missing evidence
Ordered MVCC and log primitives	[Verified]	Deterministic semantics, replay, branches, Tetris and Chess state histories	distributed integration and compatibility policy
RocksDB RangeEngine	[Verified] scoped	matched single-range reads at 1, 8, and 32 clients on GCP local NVMe	writes, eviction curve, range movement, and full cell RPC
Native quorum *txLog*	[Evaluating]	three real TCP processes on one host, restart, torn-tail repair, epoch fencing	three independent machines, matched durable control, integrated transaction
Object publication and recovery	[Verified] scoped	generation-pinned GCS root, exact closure reconstruction, corruption rejection	sustained debt, brownout, compaction amplification, branch reuse
C5v2 object layout	[Evaluating]	point and scan GCS preflight, complete recovery	sealed full-curve verdict and write path
Cell transaction plane	[Proposed]	executable safety model and component protocols	complete resolver, proxy, assignment, and multi-range runtime
RAM RangeEngine	[Proposed]	local playground replay only	matched Garnet, SSD, recovery, memory, and cost curves
PostgreSQL, Redis, HTAP	[Proposed] / [Evaluating]	isolated log primitives and bounded DataFusion overlay	end-to-end engines and workload admissions

8.3 Cache-tail diagnostic

A later ten-position provider-v2 diagnostic measured native-to-control p50, p95, p99, and p99.9 ratios of 1.026x, 1.131x, 1.742x, and 1.032x.

The p99 result misses the 1.20 gate. Because adjacent percentiles and the resident-footprint checks passed, the program records this as an explicit tail-attribution debt and moves to higher-value system gates. It is not an admitted cache curve.

8.4 Object layout and recovery

The C5v2 GCS viewer-only preflight measured 0.869x the C0 point-read p99, 31.692x the C0 projection-scan throughput, and 1.043x its object bytes. A larger curve completed all positions but its evaluator-only replay names did not match, so no sealed curve verdict is claimed.

Complete-child recovery fetched the typed root and four children using five full GETs, 13,700,110 response bytes, zero range GETs, zero LIST, and zero writes. It verified 792 projection and 792 payload proofs, reconstructed 25,014 MVCC records and 15,742 live rows, and completed in 792.221 ms. A cloud poison changed byte zero of the exact projection object and recovery rejected it at the generation-pinned child SHA boundary after the same five-GET graph.

8.5 Performance matrix

Table 4 is the program's rowing machine. Each row admits one additional system claim. Work moves upward only when the current row exposes no more important blocker than the next row.

9 ARCHITECTURAL DECISIONS AND TRADEOFFS

9.1 Why not an object-only WAL?

Object-gated commits simplify the durability story but place object latency, availability, request throttling, and often single-region assumptions directly on every acknowledgement. Batching restores throughput at the cost of latency and leader machinery. The default objectKV profile instead uses a stable quorum for the foreground suffix and asynchronous immutable publication. This adds a log service and objectification debt, but makes that debt visible and bounded. An object-committed profile remains possible when its latency and regional loss contract is acceptable.

9.2 Why not FoundationDB plus objects?

FoundationDB already supplies a mature transaction plane and remains the strict-serializability oracle and fallback. Pairing it with immutable object history is the fastest route to higher-level product learning. It also retains a second permanent storage system, keeps placement and recovery coupled to the FoundationDB fleet, and cannot directly make the open object closure the reconstruction boundary. The program therefore keeps two lanes: use FoundationDB to validate semantics and applications, while the native lane must earn replacement through matched durability and performance gates.

9.3 Why not rebuild TiKV?

Recreating TiKV's full Raft, RocksDB, placement, backup, and SQL integration stack would spend most effort reproducing a mature permanent-replica system. objectKV reuses RocksDB today and studies a narrower native log later. Its novel claim is not a faster RocksDB wrapper. It is bounded disposable range state over generation-pinned immutable history, with exact row and column views. If local bytes still scale with database bytes or recovery requires full replica copy, the distinction disappears and TiKV is the stronger substrate.

9.4 Raft or Paxos?

The product contract depends on quorum intersection, fencing, ordered decisions, and recovery generations, not algorithm branding. Raft provides an understandable initial implementation and strong library support [12]. A Paxos-family protocol may later improve reconfiguration or availability topology. The first three-machine eval should compare a correct, matched durable path before optimizing consensus variants.

9.5 RocksDB or a native hybrid log?

RocksDB is the admitted SSD provider and a durable engineering shortcut. It brings ordered reads, compaction, bloom filters, cache management, snapshots, and recovery, but can duplicate objectKV's own MVCC, log, and objectification work. A Garnet-like native hybrid log could unify RAM and NVMe serving, reduce copies, and make checkpoint phases explicit. It also requires substantial concurrency, index, eviction, and recovery engineering. The native provider should begin only with a direct-execution versus dispatch benchmark and a crash matrix for semantic versus physical logging.

9.6 Row, page, or column?

The kernel remains value-native. PostgreSQL can initially treat values as pages, preserving its buffer manager and page semantics rather than rewriting the database as a row engine. Redis-like structures and logs can store operation or object records. C5v2 can derive a scan-efficient projection from the same commit history. No single physical granularity must dominate every surface. The strict requirements are one transaction history, exact version alignment, and explicit amplification.

10 RISKS AND NEXT EVALUATIONS

The immediate sequence is narrow:

(1) finish C5v2 compaction-amplification and branch-reference measurements;
(2) regenerate a current-model staged implementation trace;
(3) run the native *txLog* on three independent NVMe machines against one matched remote durable control;
(4) integrate one transaction path only if that curve clears;
(5) then begin the first multi-range strict-serializability history.

This order avoids building PostgreSQL, Redis, and DataFusion product surfaces on an unmeasured commit and range foundation. It also avoids waiting for every tail optimization before testing the next architectural mortality risk.

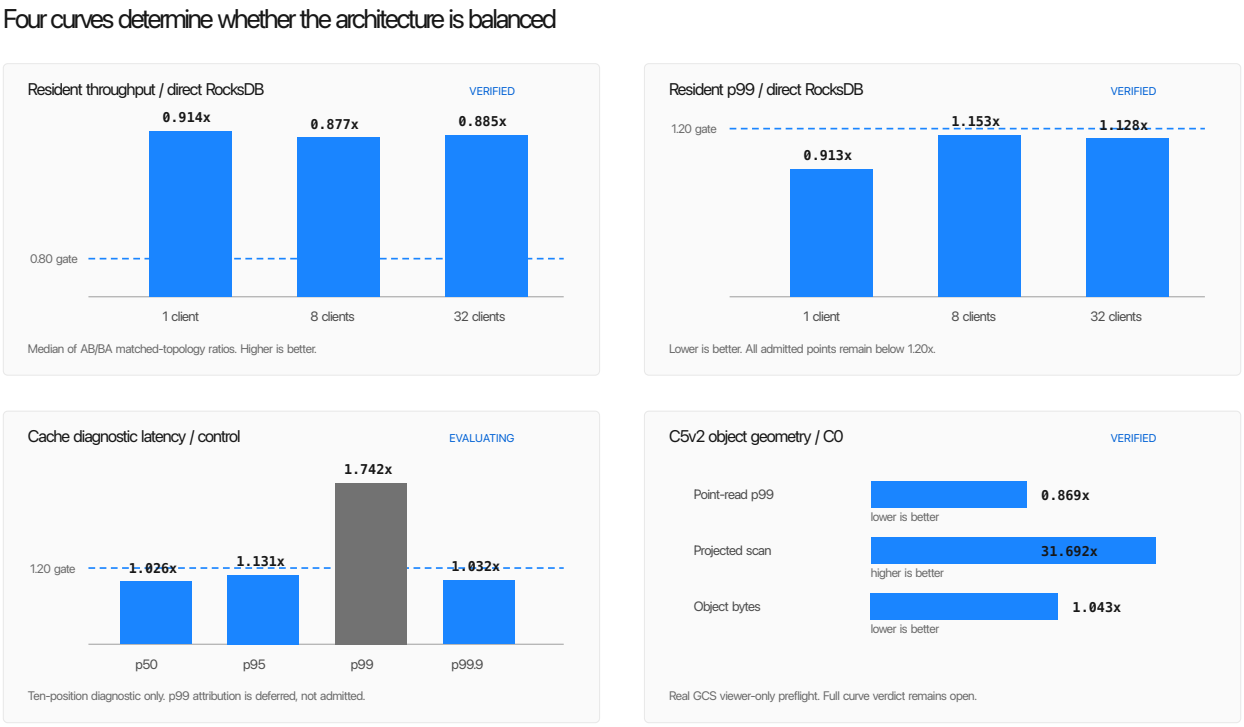

Figure 6: Four currently decision-bearing curves. Each panel has its own workload and status.

Table 4: Master performance matrix, 2026-08-30.

Row	Curve	Status	Strongest current evidence	Next decision-bearing gate
0	Resident NVMe point read	[Verified]	0.873x to 0.920x RocksDB throughput; p99 0.913x to 1.184x	retain as regression control
1	Cache coverage and eviction	[Evaluating]	footprint corrected; diagnostic p99 1.742x	attribute hit and miss samples before resuming frozen sweep
2	Cold indexed point read	[Evaluating]	one bounded range GET; provider tail missed two block gates	return after object-layout write path
3	Point and scan object geometry	[Evaluating]	C5v2 preflight and complete recovery verified	compaction amplification and branch-reference reuse
4	Native three-node commit	[Evaluating]	one-host TCP log mechanics and finite TLA+ safety	three independent NVMe machines versus matched control
5	Brownout and host loss	[Evaluating]	exact base plus suffix recovery	sustained write with object-fault schedule and hard debt ceiling
6	Branch and lazy reopen	[Evaluating]	exact local branch and empty worker	parent-size sweep with GCS request accounting
7	Multi-range cell	[Proposed]	no admitted runtime	strict-serializable cross-range history, then 1, 2, 4, 8 range groups
8	RAM serving and handoff	[Proposed]	playground semantics only	require a 20% gain on one declared end-to-end metric
9	Log, Redis, search, and VFS fabric	[Proposed]	local okv-log game histories	one semantic oracle and one specialist curve per surface
10	PostgreSQL page OLTP	[Proposed]	page boundary documented	first adapter within 2x local PostgreSQL, then resident target within 1.25x
11	DataFusion exact HTAP	[Evaluating]	exact bounded overlay; isolated source 2.544M rows/s	tails at 0, 0.1, 1, and 10% plus OLTP interference
12	Complete-stack economics	[Proposed]	component metrics only	matched failure, capacity, request, compute, and operator cost

11 RELATED WORK

FoundationDB is the primary semantic reference for ordered transactions, versioned reads, proxies, resolvers, logs, and bounded transaction domains [20]. TiDB and TiKV demonstrate the mature alternative of Raft-backed RocksDB ranges with SQL and analytical replicas [8]. Aurora, Socrates, and Neon separate database compute, durable logging, and remote storage at page and log boundaries [1, 11, 18].

Garnet and Tsavorite show how a narrow operation waist, direct request execution, typed views, hybrid logs, and explicit commit depth can produce a high-performance memory and SSD engine [5]. CloudJump III studies RAM, SSD, durable, and object tiers

Table 5: Primary architectural risks and the earliest useful measurement.

Risk	Failure signal	Earliest decisive eval	Consequence
Quorum tax	native commit p99 exceeds 1.25x matched durable control after one optimization cycle	three independent NVMe machines, exact same durability and client path	retain FoundationDB or TiKV transaction plane
Double logging	engine WAL, *txLog*, and objectification dominate bytes or CPU	per-layer bytes and CPU during mixed writes	native hybrid log or remove redundant durability layer
Object debt	$C-O$, log bytes, or recovery time grows without bound	sustained writer with 30-minute object brownout	hard backpressure, more materializers, or reject workload class
Cold-read cliff	requests or metadata bytes grow with total database size	10 GiB and 10 M key point and reopen sweep	redesign index and closure
RAM has no role	no 20% gain over admitted SSD on latency, throughput, or CPU after cost normalization	matched Garnet, RAM, and RocksDB subset	keep RAM as cache only
HTAP tail cliff	1% tail adds more than 20% or requires unbounded memory	exact DataFusion tail sweep with update invalidations	increase materialization rate or narrow zero-ETL claim
Operational overreach	repair needs full-cell copy or coupled global coordination	host-loss, branch, and range-move fault schedule	shrink cell envelope and simplify authorities

for cloud database execution [6]. HANA demonstrates transaction processing over columnar state and piecewise access [15, 16]. Paimon and RisingWave explore object-backed primary-key and state-store layouts [2, 3, 14]. objectKV differs in the proposed combination of one bounded strict-serializable cell, disposable range serving, an authenticated immutable reconstruction base, and exact row-column version alignment.

12 CONCLUSION

The emerging objectKV vision is a small transactional kernel for data platforms, not a database product with every feature embedded in the kernel. One cell owns transactionality. One stable *txLog* owns the acknowledged suffix. One RangeEngine state exposes several typed fabric views. Immutable object closures own portable reconstruction, history, branching, and analytical layout.

Current evidence supports the single-range RocksDB read boundary, scoped C5v2 GCS geometry and recovery, and the finite cell safety model. It does not yet support the complete distributed claim. The next useful proof is physical: a three-machine stable-quorum curve with exact durability and failure controls. If that path clears, multi-range transactions become the next kernel milestone. PostgreSQL, Redis-like objects, logs, search, virtual filesystems, and DataFusion then become typed consumers of one measured fabric rather than separate storage systems.

REFERENCES

[1] Panagiotis Antonopoulos, Alex Budovski, Cristian Diaconu, Alejandro Hernandez Saenz, Jack Hu, Hanuma Kodavalla, Donald Kossmann, Sandeep Lingam, Dinesh Nica, Parikshit Purohit, Peter Qu, Chaitanya Ramesh, Ashay Ravi, Cristian Simion, et al. 2019. Socrates: The New SQL Server in the Cloud. In *Proceedings of the 2019 International Conference on Management of Data*. 1743–1756. https://doi.org/10.1145/3299869.3314047

[2] Apache Software Foundation. 2024. PIP-25: Introduce a Key-Value File Format for Paimon Primary Key Table. https://cwiki.apache.org/confluence/pages/viewpage.action?pageId=311628201

[3] Apache Software Foundation. 2026. Apache Paimon Primary Key Table Overview. https://paimon.apache.org/docs/1.1/primary-key-table/overview/

[4] Apache Software Foundation. 2026. Apache Parquet Format. https://parquet.apache.org/docs/file-format/

[5] Badrish Chandramouli, Vasileios Zois, Ted Hart, Tal Zaccai, Lukas M. Maas, Yoganand Rajasekaran, and Darren Gehring. 2025. Garnet: A Next-Generation Cache-Store for Accelerating Applications and Services. *Proceedings of the VLDB Endowment* 19, 2 (2025), 224–237. https://www.vldb.org/pvldb/vol19/p224-chandramouli.pdf

[6] Zongzhi Chen, Mo Sha, Feifei Li, Sheng Wang, Baolin Huang, Guoqing Ma, Huaxiong Song, Ke Yu, Xizhe Zhang, and Yuan Wang. 2026. CloudJump III: Optimizing Cloud Databases for Tiered Storage. In *Companion of the International Conference on Management of Data*. 266–280. https://doi.org/10.1145/3788853.3803084

[7] Siying Dong, Mark Callaghan, Leonidas Galanis, Dhruba Borthakur, Tony Savor, and Michael Strum. 2017. Optimizing Space Amplification in RocksDB. In *CIDR*. https://www.cidrdb.org/cidr2017/papers/p82-dong-cidr17.pdf

[8] Dongxu Huang, Qi Liu, Qiu Cui, Zhuhe Fang, Xiaoyu Ma, Fei Xu, Li Shen, Liu Tang, Yuxing Zhou, Menglong Huang, Wan Wei, et al. 2020. TiDB: A Raft-based HTAP Database. *Proceedings of the VLDB Endowment* 13, 12 (2020), 3072–3084. https://doi.org/10.14778/3415478.3415535

[9] Aviral Khurana et al. 2024. Apache Arrow DataFusion: A Fast, Embeddable, Modular Analytic Query Engine. In *Companion of the 2024 International Conference on Management of Data*. https://dl.acm.org/doi/10.1145/3626246.3653368

[10] Wes McKinney Li et al. 2016. Apache Arrow: A Cross-Language Development Platform for In-Memory Data. In *Proceedings of the VLDB Endowment*. https://arrow.apache.org/

[11] Neon Contributors. 2022. Neon Architecture. Neon Documentation. https://neon.tech/docs/introduction/architecture-overview

[12] Diego Ongaro and John Ousterhout. 2014. *In Search of an Understandable Consensus Algorithm*. Technical Report. USENIX Association. https://raft.github.io/raft.pdf

[13] Weston Pace, Chang She, Lei Xu, Will Jones, Albert Lockett, Jun Wang, and Raunak Shah. 2025. Lance: Efficient Random Access in Columnar Storage through Adaptive Structural Encodings. *arXiv preprint arXiv:2504.15247* (2025). https://arxiv.org/abs/2504.15247

[14] RisingWave Labs. 2026. RisingWave Storage Overview. https://docs.risingwave.com/store/overview

[15] Reza Sherkat, Christian Florendo, Mihnea Andrei, Young-Seok Choi, Guk-Han Lee, Carsten Lemke, Nitin Pathak, Somdip Saha, Natalia Sokolova, Anatoly Stetsenko, et al. 2019. Page As You Go: Piecewise Columnar Access in SAP HANA. *Proceedings of the VLDB Endowment* 12, 12 (2019), 2047–2058. https://www.vldb.org/pvldb/vol12/p2047-sherkat.pdf

[16] Vishal Sikka, Franz F"arber, Wolfgang Lehner, Sang Kyun Cha, Thomas Peh, and Christof Bornh"ovd. 2012. Efficient Transaction Processing in SAP HANA Database: The End of a Column Store Myth. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*. 731–742. https://doi.org/10.1145/2213836.2213946

[17] Nicolae Vartolomei. 2026. Oswald: TLA+ Specifications for an Object-Storage Database. https://nvartolomei.com/oswald/

[18] Alexandre Verbitski, Anurag Gupta, Debanjan Saha, Murali Brahmadesam, Kamal Gupta, Raman Mittal, Sailesh Krishnamurthy, Sandor Maurice, Tengiz Kharatishvili, and Xiaofeng Bao. 2017. Amazon Aurora: Design Considerations for High Throughput Cloud-Native Relational Databases. In *Proceedings of the 2017 ACM International Conference on Management of Data*. 1041–1052. https://doi.org/10.1145/3035918.3056101

[19] Vortex Contributors. 2026. Vortex File Format. https://docs.vortex.dev/concepts/file-format

[20] Jingyu Zhou, Meng Xu, Alexander Shraer, Bala Namasivayam, Alex Miller, Evan Tschannen, Steve Atherton, Andrew J. Beamon, Rusty Sears, John Leach, Dave Rosenthal, Xin Dong, Will Wilson, Ben Collins, David Scherer, Alec Grieser, Young Liu, Alvin Moore, Bhaskar Muppana, Sampath Raghunath, Anirudh Rao, et al. 2021. FoundationDB: A Distributed Unbundled Transactional Key Value Store. In *Proceedings of the 2021 International Conference on Management of Data*. 2653–2666. https://doi.org/10.1145/3448016.3457559